JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LinkedHashSet<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractSet<E>
      java.util.HashSet<E>
        java.util.LinkedHashSet<E>

Type Parameters:

E - the type of elements maintained by this set

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, SequencedCollection<E>, SequencedSet<E>, Set<E>

```
public class LinkedHashSet<E>
extends HashSet<E>
implements SequencedSet<E>, Cloneable, Serializable
```

Hash table and linked list implementation of the `Set` interface, with well-defined encounter order. This implementation differs from `HashSet` in that it maintains a doubly-linked list running through all of its entries. This linked list defines the encounter order (iteration order), which is the order in which elements were inserted into the set (*insertion-order*). The least recently inserted element (the eldest) is first, and the youngest element is last. Note that encounter order is *not* affected if an element is *re-inserted* into the set with the `add` method. (An element `e` is reinserted into a set `s` if `s.add(e)` is invoked when `s.contains(e)` would return `true` immediately prior to the invocation.) The reverse-ordered view of this set is in the opposite order, with the youngest element appearing first and the eldest element appearing last. The encounter order of elements already in the set can be changed by using the `addFirst` and `addLast` methods.

This implementation spares its clients from the unspecified, generally chaotic ordering provided by `HashSet`, without incurring the increased cost associated with `TreeSet`. It can be used to produce a copy of a set that has the same order as the original, regardless of the original set's implementation:

```
    void foo(Set<String> s) {
        Set<String> copy = new LinkedHashSet<>(s);
        ...
    }
```

This technique is particularly useful if a module takes a set on input, copies it, and later returns results whose order is determined by that of the copy. (Clients generally appreciate having things returned in the same order they were presented.)

This class provides all of the optional `Set` and `SequencedSet` operations, and it permits null elements. Like `HashSet`, it provides constant-time performance for the basic operations (`add`, `contains` and `remove`), assuming the hash function disperses elements properly among the buckets. Performance is likely to be just slightly below that of `HashSet`, due to the added expense of maintaining the linked list, with one exception: Iteration over a `LinkedHashSet` requires time proportional to the *size* of the set, regardless of its capacity. Iteration over a `HashSet` is likely to be more expensive, requiring time proportional to its *capacity*.

A linked hash set has two parameters that affect its performance: *initial capacity* and *load factor*. They are defined precisely as for `HashSet`. Note, however, that the penalty for choosing an excessively high value for initial capacity is less severe for this class than for `HashSet`, as iteration times for this class are unaffected by capacity.

Note that this implementation is not synchronized. If multiple threads access a linked hash set concurrently, and at least one of the threads modifies the set, it *must* be synchronized externally. This is typically accomplished by synchronizing on some object that naturally encapsulates the set. If no such object exists, the set should be "wrapped" using the `Collections.synchronizedSet` method. This is best done at creation time, to prevent accidental unsynchronized access to the set:

```
  Set s = Collections.synchronizedSet(new LinkedHashSet(...));
```

The iterators returned by this class's `iterator` method are *fail-fast*: if the set is modified at any time after the iterator is created, in any way except through the iterator's own `remove` method, the iterator will throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*

This class is a member of the Java Collections Framework.

Since:

1.4

See Also:

`Object.hashCode()`, `Collection`, `Set`, `HashSet`, `TreeSet`, `Hashtable`, Serialized Form

Constructor Summary

Constructors

Constructor	Description
`LinkedHashSet()`	Constructs a new, empty linked hash set with the default initial capacity (16) and load factor (0.75).
`LinkedHashSet(int initialCapacity)`	Constructs a new, empty linked hash set with the specified initial capacity and the default load factor (0.75).
`LinkedHashSet(int initialCapacity, float loadFactor)`	Constructs a new, empty linked hash set with the specified initial capacity and load factor.
`LinkedHashSet(Collection<? extends E> c)`	Constructs a new linked hash set with the same elements as the specified collection.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`void`	`addFirst(E e)`	Adds an element as the first element of this collection (optional operation).
`void`	`addLast(E e)`	Adds an element as the last element of this collection (optional operation).
`E`	`getFirst()`	Gets the first element of this collection.
`E`	`getLast()`	Gets the last element of this collection.

static <T> LinkedHashSet<T>	**newLinkedHashSet**(int numElements)	Creates a new, empty LinkedHashSet suitable for the expected number of elements.
E	**removeFirst**()	Removes and returns the first element of this collection (optional operation).
E	**removeLast**()	Removes and returns the last element of this collection (optional operation).
SequencedSet<E>	**reversed**()	Returns a reverse-ordered view of this collection.
Spliterator<E>	**spliterator**()	Creates a *late-binding* and *fail-fast* Spliterator over the elements in this set.

Methods declared in class HashSet

add, clear, clone, contains, isEmpty, iterator, newHashSet, remove, size, toArray, toArray

Methods declared in class AbstractSet

equals, hashCode, removeAll

Methods declared in class AbstractCollection

addAll, containsAll, retainAll, toArray, toArray, toString

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Collection

parallelStream, removeIf, stream, toArray

Methods declared in interface Iterable

`forEach`

Methods declared in interface Set

`addAll`, `containsAll`, `retainAll`

Constructor Details

LinkedHashSet

```
public LinkedHashSet(int initialCapacity,
                     float loadFactor)
```

Constructs a new, empty linked hash set with the specified initial capacity and load factor.

API Note:

To create a `LinkedHashSet` with an initial capacity that accommodates an expected number of elements, use `newLinkedHashSet`.

Parameters:

`initialCapacity` - the initial capacity of the linked hash set

`loadFactor` - the load factor of the linked hash set

Throws:

`IllegalArgumentException` - if the initial capacity is less than zero, or if the load factor is nonpositive

LinkedHashSet

```
public LinkedHashSet(int initialCapacity)
```

Constructs a new, empty linked hash set with the specified initial capacity and the default load factor (0.75).

API Note:

To create a `LinkedHashSet` with an initial capacity that accommodates an expected number of elements, use `newLinkedHashSet`.

Parameters:

`initialCapacity` - the initial capacity of the LinkedHashSet

Throws:

`IllegalArgumentException` - if the initial capacity is less than zero

LinkedHashSet

```
public LinkedHashSet()
```

Constructs a new, empty linked hash set with the default initial capacity (16) and load factor (0.75).

LinkedHashSet

```
public LinkedHashSet(Collection<? extends E> c)
```

Constructs a new linked hash set with the same elements as the specified collection. The linked hash set is created with an initial capacity sufficient to hold the elements in the specified collection and the default load factor (0.75).

Parameters:

`c` - the collection whose elements are to be placed into this set

Throws:

`NullPointerException` - if the specified collection is null

Method Details

spliterator

```
public Spliterator<E> spliterator()
```

Creates a *late-binding* and *fail-fast* `Spliterator` over the elements in this set.

The `Spliterator` reports `Spliterator.SIZED`, `Spliterator.DISTINCT`, and `ORDERED`. Implementations should document the reporting of additional characteristic values.

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Set<E>`

Overrides:

`spliterator` in class `HashSet<E>`

Implementation Note:

The implementation creates a *late-binding* spliterator from the set's `Iterator`. The spliterator inherits the *fail-fast* properties of the set's

iterator. The created `Spliterator` additionally reports `Spliterator.SUBSIZED`.

Returns:

a `Spliterator` over the elements in this set

Since:

1.8

newLinkedHashSet

```
public static <T> LinkedHashSet<T> newLinkedHashSet(int numElements)
```

Creates a new, empty LinkedHashSet suitable for the expected number of elements. The returned set uses the default load factor of 0.75, and its initial capacity is generally large enough so that the expected number of elements can be added without resizing the set.

Type Parameters:

`T` - the type of elements maintained by the new set

Parameters:

`numElements` - the expected number of elements

Returns:

the newly created set

Throws:

`IllegalArgumentException` - if numElements is negative

Since:

19

addFirst

```
public void addFirst(E e)
```

Adds an element as the first element of this collection (optional operation). After this operation completes normally, the given element will be a member of this collection, and it will be the first element in encounter order.

If this set already contains the element, it is relocated if necessary so that it is first in encounter order.

Specified by:

`addFirst` in interface `SequencedCollection<E>`

Parameters:

`e` - the element to be added

Since:

21

addLast

```
public void addLast(E e)
```

Adds an element as the last element of this collection (optional operation). After this operation completes normally, the given element will be a member of this collection, and it will be the last element in encounter order.

If this set already contains the element, it is relocated if necessary so that it is last in encounter order.

Specified by:

addLast in interface SequencedCollection<E>

Parameters:

e - the element to be added.

Since:

21

getFirst

```
public E getFirst()
```

Gets the first element of this collection.

Specified by:

getFirst in interface SequencedCollection<E>

Returns:

the retrieved element

Throws:

NoSuchElementException - if this collection is empty

Since:

21

getLast

```
public E getLast()
```

Gets the last element of this collection.

Specified by:

getLast in interface SequencedCollection<E>

Returns:

the retrieved element

Throws:

NoSuchElementException - if this collection is empty

Since:

21

removeFirst

```
public E removeFirst()
```

Removes and returns the first element of this collection (optional operation).

Specified by:

removeFirst in interface SequencedCollection<E>

Returns:

the removed element

Throws:

NoSuchElementException - if this collection is empty

Since:

21

removeLast

```
public E removeLast()
```

Removes and returns the last element of this collection (optional operation).

Specified by:

removeLast in interface SequencedCollection<E>

Returns:

the removed element

Throws:

NoSuchElementException - if this collection is empty

Since:

21

reversed

```
public SequencedSet<E> reversed()
```

Returns a reverse-ordered view of this collection. The encounter order of elements in the returned view is the inverse of the encounter order of elements in this collection. The reverse ordering affects all order-sensitive operations, including those on the view collections of the returned view. If the collection implementation permits modifications to this view, the

modifications "write through" to the underlying collection. Changes to the underlying collection might or might not be visible in this reversed view, depending upon the implementation.

Modifications to the reversed view are permitted and will be propagated to this set. In addition, modifications to this set will be visible in the reversed view.

Specified by:

`reversed` in interface `SequencedCollection<E>`

Specified by:

`reversed` in interface `SequencedSet<E>`

Returns:

a reverse-ordered view of this collection, as a `SequencedSet`

Since:

21

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.